

Name **Khuram ullah khan**

Reg No **L1F24BSAI0038**

Subject **AI Lab**

Section **AD1**

Lab work **week 3**

Task 1:

Code:

```
import time

def time_it(func):
    def wrapper():
        start = time.perf_counter()
        result = func()
        end = time.perf_counter()
        print("the execution Time is:", end - start, "seconds")
        return result
    return wrapper

@time_it
def sum_of_squares():
    total = 0
    for i in range(1, 1000000):
        total += i * i
    return total

answer = sum_of_squares()

print("the sum of Squares =", answer)
```

output:

```
        print("the execution Time is: ", end = start, sep = ", ")
        return result
    return wrapper

@time_it
def sum_of_squares():
    total = 0
    for i in range(1, 1000000):
        total += i * i
    return total

answer = sum_of_squares()
print("the sum of Squares =", answer)

the execution Time is: 0.0791873229999851 seconds
the sum of Squares = 333332833333500000
```

Task 2:

Code:

```
students = [  
    ("khuram", 81, 18),  
    ("Soffi", 89, 21),  
    ("miksi", 76, 16),  
    ("zaki", 63, 19),  
    ("salman", 93, 15)  
]  
  
sorted_students = sorted(students, key=lambda x: (-x[1], x[2]))  
  
print("The Sorted Student List is:")  
  
for student in sorted_students:  
    print(student)
```

output:

```
▶ students = [  
    ("khuram", 81, 18),  
    ("soffi", 89, 21),  
    ("miksi", 76, 16),  
    ("zaki", 63, 19),  
    ("salman", 93, 15)  
]  
sorted_students = sorted(students, key=lambda x: (-x[1], x[2]))  
print("The Sorted Student List is:")  
for student in sorted_students:  
    print(student)
```

```
... The Sorted Student List is:  
( 'salman', 93, 15)  
( 'soffi', 89, 21)  
( 'khuram', 81, 18)  
( 'miksi', 76, 16)  
( 'zaki', 63, 19)
```

Task 3:

Code:

```
def fibonacci(n):  
    a = 0  
    b = 1  
    for i in range(n):  
        yield a  
        a, b = b, a + b  
print("First 100 Fibonacci Numbers:")  
for number in fibonacci(100):  
    print(number)
```

output:

```
... 267914296
    433494437
    701408733
    1134903170
    1836311903
    2971215073
    4807526976
    7778742049
    12586269025
    20365011074
    32951280099
    53316291173
    86267571272
    139583862445
    225851433717
    365435296162
    591286729879
    956722026041
    1548008755920
    2504730781961
    4052739537881
    6557470319842
    10610209857723
    17167680177565
    27777890035288
    44945570212853
    72723460248141
    117669030460994
    190392490709135
    308061521170129
    498454011879264
    806515533049393
```

Task 4:

Code:

```
class SecureFileManager:
    def __init__(self, filename):
        self.filename = filename
    def __enter__(self):
        self.file = open(self.filename, "w")
        return self
    def write(self, text):
        self.file.write(text.upper())
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()
        print("File closed successfully")
with SecureFileManager("output.txt") as f:
    f.write("hello world\n")
    f.write("python programming")
```

output:

```
▶ class SecureFileManager:
    def __init__(self, filename):
        self.filename = filename
    def __enter__(self):
        self.file = open(self.filename, "w")
        return self
    def write(self, text):
        self.file.write(text.upper())
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()
        print("File closed successfully")
with SecureFileManager("output.txt") as f:
    f.write("hello world\n")
    f.write("python programming")

... File closed successfully
```

Task 5:

1. Closure in Decorator

A closure means the inner function can remember variables from the outer function. In decorators, the wrapper function remembers the func even after the decorator finishes. This allows the wrapper to call the original function later.

2. Lambda vs Normal Function

A lambda function is a small one-line function that contains only one expression and returns its result automatically. A normal function created with def can have multiple statements and uses the return keyword.

3. Yield in Generator

When yield is used, the function returns a value but pauses instead of stopping completely. It saves its variables and position so it can continue from the same place later. This is different from return, which ends the function.

4. Exception Handling in Context Manager

In a context manager, the `__exit__()` method handles exceptions that happen inside the with block. It receives the error information and can decide whether to stop the error or let it continue.